Foundations of Agentic AI for Science & Engineering

AESCAPE 2026 · SC03: Introduction to Agentic Workflows

Nathaniel Trask

University of Pennsylvania

September 8, 2026

Today

1. **Foundations.** What an agent is. Tools, schemas, the loop. Build ReAct from scratch in ∼30 lines.
2. **The framework zoo.** ReAct, graphs (LangGraph), conversational multi-agent (AutoGen), MPC-style planning — and MCP. *Syntax, a toy, and an exemplar for each.*
3. **Scientific computing as a tool surface.** CAD → mesh → solve → post, re-cut so an LLM can drive it.
4. **Reliability.** Verifier agents, manufactured solutions, oracles.

The one question this talk is organized around

For any given problem, **which agentic pattern should you reach for, and why?** Not "how do I use library X."

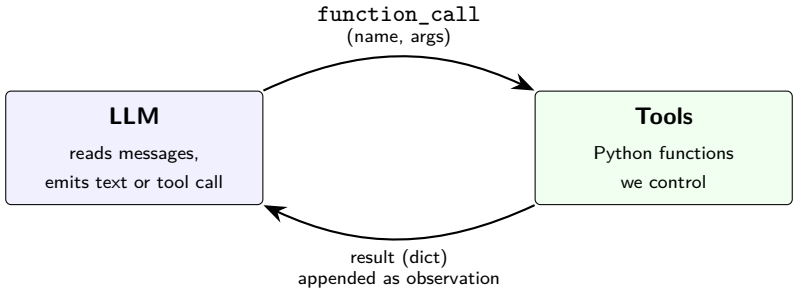
What is an LLM agent?

Agent = LLM + tools + a loop.

- A bare LLM takes text in, emits text out. One shot.
- An **agent** is an LLM running in a loop with access to **tools**: ordinary Python functions it can ask us to run on its behalf.
- Each turn, the model reads the running conversation and decides to either
  - emit a final text answer, or
  - call a tool with structured arguments.
- When it calls a tool, *we* run the function and feed the result back as the next turn.

The model never executes anything. It emits a request; you stay in control of what actually runs.

The loop, as a picture



when the model stops calling tools, it emits a final text answer and the loop exits

This is **ReAct** (Reason + Act), Yao et al. 2022 — the simplest agent there is.

What a tool is

Two pieces, both written by you: a Python function, and a JSON-schema description of its signature that you hand to the model.

Python function (you call it)

```python
def calculator(op, a, b):
    ops = {"add": a + b, "sub": a - b,
           "mul": a * b, "div": a / b}
    return {"result": ops[op]}
```

JSON schema (the model reads it)

```json
{"name": "calculator",
 "description": "One arithmetic op.",
 "parameters": {
   "type": "object",
   "properties": {
     "op": {"type": "string",
            "enum": ["add","sub","mul","div"]},
     "a": {"type": "number"},
     "b": {"type": "number"}},
   "required": ["op","a","b"]}}
```

- `name`, `description` — decide whether the model picks *this* tool over the others. This is prompt engineering; write it carefully.
- `properties` — the typed argument list. `enum` restricts a string to a fixed set. Constrain aggressively: every value you forbid in the schema is a failure mode you never have to debug.

The round trip

Each turn, the model emits either plain text (final answer) or a structured tool call.

1. The reply contains a `function_call` part:

```
response.candidates[0].content.parts[0].function_call
    .name = "calculator"
    .args = {"op": "mul", "a": 13, "b": 47}
```

2. We dispatch by name and run the Python function:

```
result = tool_fns[name](**args)          # -> {"result": 611}
```

3. We send the result back as a `function_response` on the next turn:

```
Part.from_function_response(name="calculator", response={"result": 611})
```

The model then sees its own call, the result, and decides whether to call another tool or stop.

The whole thing, in pseudocode

```
def run_agent(system_prompt, user_prompt, tool_schemas, tool_fns, max_steps=12):
    messages = [user_prompt]
    for step in range(max_steps):
        response = llm(system_prompt, messages, tools=tool_schemas)
        messages.append(response)

        if response.is_tool_call:
            for call in response.tool_calls:
                try:
                    result = tool_fns[call.name](**call.args)
                except Exception as e:
                    result = {"error": str(e)}      # errors are observations
                messages.append(result)
        else:
            return response.text                    # final answer
```

That is a complete agent. Everything else in this talk is a variation on **who decides what happens next**.

Two slots: `system_prompt` vs `user_prompt`

- `system_prompt` — the *rules of the game*. Identity, methodology, what the tools are for, budgets, how to stop. Sent on every turn as a persistent header; never drowned out as observations pile up.
- `user_prompt` — the *specific ask* that triggers this run. Becomes the first message; tool calls and observations append after it.

```
B1_SYSTEM = """You optimize a black-box function (angle in degrees, in (0,90))
returning a range in meters. Budget: at most 12 evaluate() calls. Use history if
helpful, then call propose_answer with your guess."""

B1_USER = "Find the optimal angle. Budget: 12 evaluations."
```

Rule of thumb: durable rules about *how to behave* go in the system prompt; the specific *thing to do this run* goes in the user prompt.

Errors are observations, not crashes

Tool calls *will* fail. The model invents an argument, asks for a mesh ID that does not exist, passes a string where you wanted a float.

```
try:
    result = tool_fns[name](**args)
except Exception as e:
    result = {"error": f"{type(e).__name__}: {e}"}
```

The model reads the error on the next turn and, with a reasonable prompt, retries with correct arguments.

You will see this happen live today

An agent recovering from its own bad tool call is the system working, not the system failing. The failure mode to actually fear is on slide “agents are confidently wrong.”

Guardrails belong in the tool, not the prompt

A prompt is a request. A tool is an enforcement point.

Hope (prompt)	**Guarantee** (tool)
"don't refine past 20k DOFs"	`if new_dofs > MAX_DOFS: return {"error": ...}`
"use at most 12 evaluations"	counter in the tool; refuse call 13
"don't touch the boundary"	tool has no boundary argument
"stop when converged"	`stop()` is the only terminal tool

Anything you would be unwilling to let a nondeterministic process do, **make impossible in the tool signature** — do not ask nicely in English.

This is the single highest-leverage habit for making agentic workflows safe enough to run unattended.

Setup: getting a key

1. **Free, no credit card.** Google AI Studio: `https://aistudio.google.com/apikey`. Sign in with any Google account. Use a project where billing has *never* been enabled.
2. **Bring your own.** Anthropic (`https://console.anthropic.com`) or OpenAI (`https://platform.openai.com/api-keys`) both work.
3. In Colab: **Secrets** panel (key icon, left sidebar), add `GEMINI_API_KEY`. Locally: `export GEMINI_API_KEY=...`
4. Run the Part 0 cells to confirm the client connects.

Never paste a key into a notebook cell

It ends up in the `.ipynb` JSON, in your shell history, and in git. Use the secrets panel or an environment variable.

The framework zoo

ReAct · LangGraph · AutoGen · MPC · MCP

Why not just ReAct?

ReAct is greedy. At each step the model sees the history and picks one action. That is exactly right when:

- the tool surface is small,
- mistakes are cheap and recoverable, and
- you cannot specify the procedure in advance.

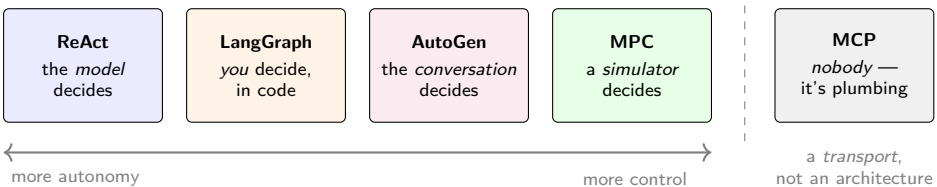
It starts to hurt when:

- you need a **guarantee** that step B always follows step A (audit, compliance, "always verify before reporting"),
- genuinely **different expertise** needs different system prompts and different tools,
- a wrong action is **expensive** and you own a cheap forward model that could have caught it, or
- your tools live in **someone else's process** — a solver, a CAD kernel, a database.

Each of those pressures produced a different pattern. That is the zoo.

The only question that matters

Choosing a pattern is choosing **who owns control flow**.



MCP sits apart on purpose: it is a *transport*, not an architecture. You can use it with any of the other four. (More on the unfortunate MPC/MCP name collision shortly.)

The zoo, on one slide

	Control flow	Lookahead	Roles	Reach for it when
ReAct	model, step by step	none	one	open-ended, errors cheap
LangGraph	you, as a graph	none	many, fixed	you need guarantees
AutoGen	emergent, via chat	none	many, fluid	expertise must collide
MPC	simulator ranks plans	N steps	one + model	wrong actions are costly
MCP	*orthogonal* — how tools reach the agent, not how it decides			

For each of the four architectures we will look at:

1. the **syntax** — what the code actually looks like,
2. a **toy** — the same trivial task, so the comparison is fair,
3. an **exemplar** — a problem where that pattern genuinely wins.

The common toy task

To compare syntax fairly, every framework gets the *same* trivial job:

Toy task

Given a target number, use a `calculator` tool to reach it, then report the result. No framework should need more than ∼20 lines.

The toy is deliberately too easy to justify any of them. That is the point: when the task is trivial, all four look like unnecessary ceremony, and you can see the raw syntax without the problem getting in the way.

Then each **exemplar** shows the task where the ceremony pays for itself.

Notebook: `01_agentic_patterns.ipynb`, §1–§5

Pattern 1 — ReAct

the model owns control flow

ReAct — syntax

No framework. A `for` loop and a dispatch table.

```
contents = [user_prompt]
for step in range(max_steps):
    resp = client.models.generate_content(
        model=MODEL, contents=contents,
        config=GenerateContentConfig(
            system_instruction=system_prompt,
            tools=[Tool(function_declarations=tool_schemas)]))
    contents.append(resp.candidates[0].content)

    calls = [p.function_call for p in resp.candidates[0].content.parts
             if p.function_call]
    if not calls:
        return resp.text                        # final answer

    for fc in calls:
        result = tool_fns[fc.name](**dict(fc.args))
        contents.append(Part.from_function_response(
            name=fc.name, response=result))
```

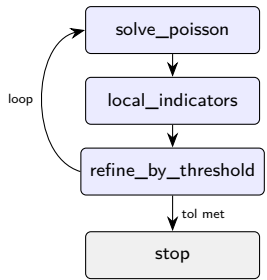
Dependencies: one SDK. State: the `contents` list. That's it.

ReAct — exemplar: adaptive mesh refinement

Why ReAct is the right call here:

- The number of refinement cycles is **not knowable in advance** — it depends on observed error.
- Every action is **cheap and recoverable**: a bad refinement just makes a mesh you discard.
- The model must **react to observations** it could not have predicted (the error estimate after each solve).
- The tool surface is **small** — six tools.

Greedy, one-step-at-a-time decision making is not a compromise here. It is the correct algorithm.



The model chooses the arrow to take at every step.

Nothing in the code forces this order.

Pattern 2 — Graphs (LangGraph)

you own control flow

LangGraph — syntax

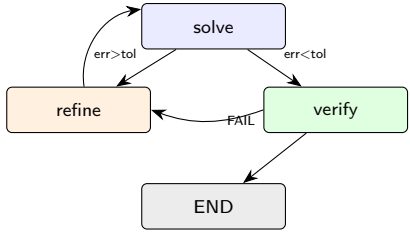
You declare a typed state, nodes that transform it, and edges. The LLM fills in nodes; *the graph* decides what runs next.

```python
from langgraph.graph import StateGraph, END
from typing import TypedDict

class S(TypedDict):
    mesh_id: int
    error: float
    attempts: int

def solve(s):  return {"error": run_solver(s["mesh_id"]), ...}
def refine(s): return {"mesh_id": do_refine(s["mesh_id"]),
                       "attempts": s["attempts"] + 1}

def route(s):                                   # <- control flow, in Python
    if s["error"] < TOL:      return "verify"
    if s["attempts"] >= 6:    return "give_up"
    return "refine"

g = StateGraph(S)
g.add_node("solve", solve); g.add_node("refine", refine)
g.add_node("verify", verify)
g.set_entry_point("solve")
g.add_conditional_edges("solve", route,
    {"refine": "refine", "verify": "verify", "give_up": END})
g.add_edge("refine", "solve")
app = g.compile()
```

LangGraph — what you actually bought

Guarantees, not suggestions:

- `verify` **always** runs before a result is reported. The model cannot decide to skip it.
- The retry count is **bounded in code**. No prompt can talk it into a seventh attempt.
- State is **typed and inspectable** — you can log, checkpoint, and resume mid-run.
- The control flow is a **diagram you can show a reviewer**, which matters when the answer feeds a design decision.



Compare with the ReAct diagram: same boxes, but here the *arrows are code*. The model never sees them.

LangGraph — exemplar: solve → verify → repair

The problem: you want an FEM result you can put in a report.

Requirement	Who enforces it
Every reported result is verified	the graph, not the prompt
At most 6 refinement attempts	the router, in Python
A failed verification triggers repair, not a retry	a conditional edge
The whole run is reproducible from a checkpoint	typed state

None of these are things you want a language model deciding on a given Tuesday. When "it usually does the right thing" is not good enough, you stop asking and start wiring.

Reach for a graph when the control flow is a requirement, not a choice.

Pattern 3 — Conversational multi-agent (AutoGen)

the conversation owns control flow

AutoGen — syntax

You define participants with distinct system prompts and let them talk. Control flow emerges from who speaks next.

```
from autogen_agentchat.agents import AssistantAgent
from autogen_agentchat.teams import RoundRobinGroupChat
from autogen_agentchat.conditions import TextMentionTermination

modeler = AssistantAgent("modeler", model_client=client,
    system_message="You propose discretizations. Be concrete and specific.")

analyst = AssistantAgent("analyst", model_client=client,
    system_message="""You are a numerical analyst. Challenge the modeler's
    proposal on stability, conditioning, and convergence rate. Do not be
    agreeable; your value is in the objection.""")

critic = AssistantAgent("critic", model_client=client,
    system_message="Judge the exchange. Say APPROVE only when convinced.")

team = RoundRobinGroupChat([modeler, analyst, critic],
    termination_condition=TextMentionTermination("APPROVE"))

await team.run(task="Choose a discretization for advection-dominated flow.")
```

AutoGen — exemplar: the design review

The problem: choosing a discretization for an advection-dominated problem. There is no single correct answer, and the *reasoning* is the deliverable.

- A single ReAct agent asked "pick a discretization" produces a confident paragraph with no adversarial pressure on it.
- Three agents with **genuinely different objectives** — propose, object, judge — produce a transcript in which the weaknesses are stated out loud.
- The artifact is not the answer. It is **the argument**, which a human can audit far faster than they can re-derive.

Reach for conversation when disagreement is the product.

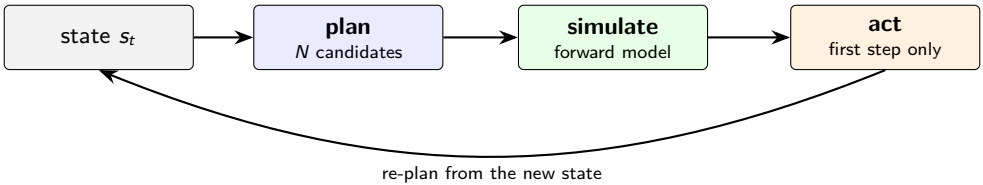
Caveat worth stating plainly: this pattern is the easiest of the four to fool yourself with. Three LLMs agreeing is not three experts agreeing — they share a prior. Distinct *tools* per role helps far more than distinct adjectives in the system prompt.

Pattern 4 — MPC-style planning

a simulator owns control flow

MPC — the idea, borrowed from control

Model Predictive Control, as you already know it:



The agentic version swaps one box: **the LLM proposes the candidate plans**, and the simulator — not the LLM — ranks them.

- The LLM contributes what it is good at: *proposing plausible candidates* from context.
- The simulator contributes what it is good at: *being right*.
- You commit only to the first action, then re-plan with real feedback.

MPC — syntax

No library needed. The pattern is the loop.

```
state = initial_state
for t in range(horizon):
    # 1. LLM proposes N candidate action sequences (cheap, creative)
    plans = llm_propose(state, history, n_candidates=5, lookahead=3)

    # 2. The *simulator* scores each one (trusted, not creative)
    scores = [world_model.rollout(state, plan) for plan in plans]

    # 3. Commit to the first action of the best plan only
    best = plans[argmax(scores)]
    state = env.step(best[0])            # <- the only real action taken

    # 4. Re-plan from the new state (closes the loop on reality)
```

Note what the LLM never does: **it never decides which plan is best**. It generates hypotheses; the forward model adjudicates. That division is the entire value of the pattern.

MPC — exemplar: black-box optimization

The problem: find the launch angle maximizing range for a projectile under quadratic drag. The agent sees only `evaluate(angle)`.

$$\ddot{x} = -c_d|v|\dot{x}, \qquad \ddot{y} = -g - c_d|v|\dot{y}, \qquad v_0 = 50\,\mathrm{m/s},\ c_d = 0.01\,\mathrm{m}^{-1}$$

- Without drag the optimum is 45°; with drag it shifts lower. The model's prior is *almost* right, which is the interesting case.
- Every `evaluate` call is a real ODE integration — the thing we are budgeting.
- We have a **cheap, trusted forward model**: the simulator itself.

In the notebook we run **ReAct vs. MPC head to head on the same budget** and count evaluations to a fixed tolerance.

This is the honest way to answer "when would I reach for one over the other" — measure it, don't assert it.

MPC — when it wins, and when it doesn't

Reach for MPC when:

- A wrong action is **expensive or irreversible** — burning budget, committing a mesh, running an experiment.
- You have a **cheap forward model you trust** more than the LLM.
- Actions compose, so looking N steps ahead genuinely differs from looking one step ahead.

Don't bother when:

- Simulating a plan costs about what executing it costs — then just execute and observe.
- You have no forward model. Then MPC degenerates into the LLM grading its own homework, which is worse than ReAct because it is *slower* and equally wrong.
- The problem is convex and small. Use `scipy.optimize` and go home.

The uncomfortable baseline

For the projectile, `minimize_scalar` finds the optimum in $\sim$12 evaluations with no LLM at all. We run it in the notebook. If an agent cannot beat `scipy`, that is worth knowing — and worth saying out loud.

Pattern 5 — MCP

not an architecture at all

MPC is not MCP

Two three-letter acronyms, one letter apart, both in this talk. Sorry.

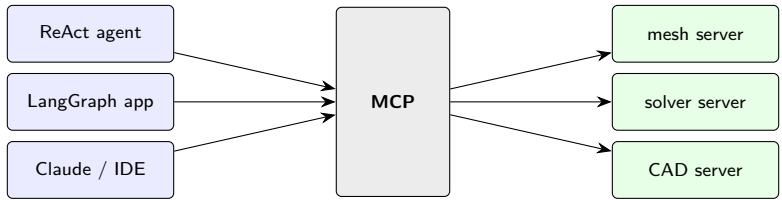
MPC — Model Predictive Control	**MCP** — Model Context Protocol
A *control strategy*.	A *wire protocol*.
Borrowed from process control, 1970s.	Introduced by Anthropic, 2024.
Answers: *how do I decide what to do next?*	Answers: *how does my agent reach a tool that lives somewhere else?*
Changes your agent's reasoning.	Changes your agent's plumbing.

They are orthogonal. You can — and often should — run an **MPC-style agent whose tools are served over MCP**.

MCP — the problem it solves

So far, every tool has been a Python function in the same process as the loop. That does not survive contact with a real simulation stack.

Your solver is: a compiled binary. On a cluster. Behind a scheduler. Written by someone else, in Fortran, in 1994. Licensed per seat.



MCP standardizes the contract between the two sides: **write your solver's tool surface once**, and any MCP-speaking client can drive it — your notebook today, a colleague's LangGraph pipeline tomorrow, a desktop assistant after that.

MCP — syntax

Server side: decorate the functions you want to expose.

```python
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("fem-tools")

@mcp.tool()
def solve_poisson(mesh_id: int) -> dict:
    """Solve -Laplacian(u) = 1 with u=0 on the boundary."""
    u, basis = solve_poisson_on(MESH_REGISTRY[mesh_id])
    return {"dofs": int(basis.N), "H1_error": h1_error(...)}

@mcp.tool()
def refine_by_threshold(mesh_id: int, theta: float = 0.5) -> dict:
    """Doerfler-mark and refine. theta in (0,1)."""
    ...

mcp.run()          # now speaks MCP over stdio or HTTP
```

The type hints and docstrings *become* the JSON schema. Notice this is the same information as slide "what a tool is" — MCP does not change what a tool is, only **who can reach it**.

Choosing: a decision guide

If this is true of your problem...	start here
I can't specify the procedure in advance, and mistakes are cheap	**ReAct**
Something must *always* happen (verify, log, approve), or a human signs off on the result	**LangGraph**
The roles need different tools and different incentives, and I want the disagreement on record	**AutoGen**
Actions are expensive or irreversible, and I own a forward model	**MPC**
My tools live in another process, language, or machine — or others need to reuse them	**MCP** (with any of the above)

Start at the top. Every row down is more machinery, more failure surface, and more code you own. Earn each step.

A word on framework enthusiasm

All four patterns are ∼30 lines of Python. The frameworks add persistence, retries, streaming, tracing, and a community — real value, at the cost of an abstraction between you and the loop.

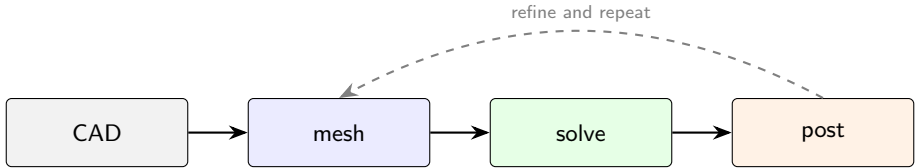
Suggested order of operations

1. Write the loop yourself, once, for your problem. It is an afternoon.
2. Find out which part actually hurts (state? retries? observability?).
3. *Then* adopt the framework that fixes that specific pain.

In today's notebook every pattern appears **twice**: once in the real framework's syntax, and once in ∼15 lines of plain Python. The second version is there to demystify the first — and so the notebook still runs when a `pip install` fails on conference wifi.

Scientific computing as a tool surface

The workflow you already know



Most of you do this daily. The question for the rest of the morning is not how it works — it is:

What does this look like when an LLM is holding the mouse?

Three things have to change:

1. Every stage needs a **JSON-callable signature**.
2. Objects that can't cross a JSON boundary (meshes, bases, matrices) need **handles**.
3. Every stage needs **guardrails**, because the caller is nondeterministic.

scikit-fem: the whole workflow in 30 lines

```python
from skfem import *
from skfem.helpers import dot, grad

@BilinearForm                          # weak form:  a(u,v) = int grad u . grad v
def stiffness(u, v, w):
    return dot(grad(u), grad(v))

@LinearForm                            # load:       L(v)   = int 1 * v
def unit_load(v, w):
    return 1.0 * v

mesh  = MeshTri.init_lshaped().refined(2)      # "CAD" + mesh
basis = Basis(mesh, ElementTriP1())            # discretization

K = stiffness.assemble(basis)                  # assemble
b = unit_load.assemble(basis)
u = solve(*condense(K, b, D=mesh.boundary_nodes()))   # solve

mesh.draw(); basis.plot(u)                     # post
```

Pure Python, pip-installable, runs in Colab. Small enough to read in one sitting, real enough to show genuine convergence behavior — which is why we use it rather than a production code.

Re-cutting the workflow as tools

The agent cannot hold a `MeshTri` object. It can hold an integer.

```
MESH_REGISTRY = {}                # int -> mesh.  The agent sees only the int.
NEXT_ID = [0]

def _register(mesh):
    mid = NEXT_ID[0]; NEXT_ID[0] += 1
    MESH_REGISTRY[mid] = mesh
    return mid

def tool_solve_poisson(mesh_id: int):
    m = MESH_REGISTRY.get(mesh_id)
    if m is None:
        return {"error": f"no mesh with id {mesh_id}"}     # observation, not crash
    u, basis = solve_poisson_on(m)
    return {"mesh_id": mesh_id, "dofs": int(basis.N),
            "H1_error": h1_error(m, u)}
```

This **handle pattern** is the single most reusable idea in the talk. Meshes, solver contexts, CAD bodies, open files, database cursors — anything with identity and no JSON representation gets an integer and a registry.

Task: adaptive refinement on the L-shape

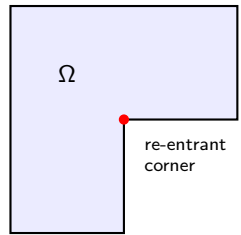
Poisson with uniform forcing:

$$-\Delta u = 1 \ \text{ in } \Omega, \qquad u = 0 \ \text{ on } \partial\Omega.$$

The re-entrant corner gives a singular solution

$$u \sim r^{2/3} \sin(2\theta/3),$$

so $u \in H^{5/3-\epsilon}$ but $u \notin H^2$.



P1 elements, expected rates:

- **Uniform** refinement: $O(h^{2/3})$ asymptotically. Pre-asymptotically you often see 0.7–0.95.
- **Adaptive** refinement: optimal $O(N^{-1/2})$, i.e. rate 1 against $h \sim N^{-1/2}$.

The gap between those two rates is the entire point of adaptive meshing. **We are going to make an agent rediscover it.**

What “adaptive” actually means

1. A local error indicator η_K. A computable proxy for how much error lives in element K. We use the gradient-jump indicator:

$$\eta_K^2 = \tfrac{1}{2} \sum_{e \in \partial K \cap \Omega^\circ} h_e \left| [\![\nabla u_h \cdot n_e]\!] \right|^2.$$

P1 gradients are discontinuous across edges, and the size of the jump correlates with local error.

2. Dörfler marking. Pick a bulk fraction $\theta \in (0, 1)$. Mark the smallest set $\mathcal{M}$ whose squared indicators capture a fraction θ of the total:

$$\sum_{K \in \mathcal{M}} \eta_K^2 \;\geq\; \theta \sum_{K} \eta_K^2.$$

Sort descending, walk the list until the running sum hits $\theta \cdot \text{total}$, refine what you walked past. $\theta = 0.5$ is the textbook default; smaller θ focuses on the worst offenders, larger θ approaches uniform refinement.

Tools the FEM agent gets

Each takes a `mesh_id` and returns a dict.

- `solve_poisson(mesh_id)` → `{dofs, H1_error}`
- `local_indicators(mesh_id)` → summary stats of η_K
- `uniform_refine(mesh_id)` → new mesh id
- `refine_by_threshold(mesh_id, theta)` → Dörfler-mark and refine
- `check_convergence_rate()` → log-log slope of the last 3 solves
- `stop(reason)`

Guardrails, in the tools:

- `MAX_DOFS = 20000` — a refinement that would exceed it returns an error observation instead of allocating.
- Every dispatch is `try/except` wrapped.
- `stop` is the only terminal tool.

Reliability

or: agents are confidently wrong

The failure mode that should actually worry you

A crashed tool call is *fine*. You see a traceback; the model retries.

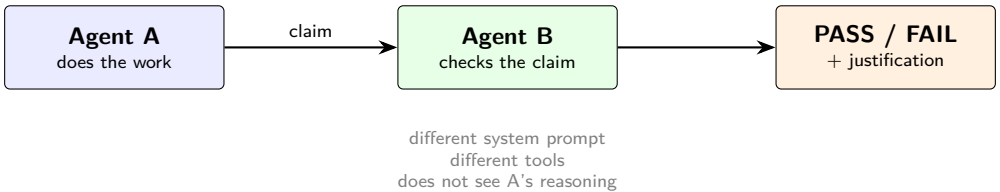
The dangerous output is a **plausible number with a confident summary** and a bug upstream of both.

In our setting, all of these produce clean-looking convergence plots:

- the agent refines toward the wrong corner and the error still decreases;
- the error *estimate* is fine but the solver has a sign error in the load;
- the reference energy is computed on too coarse a mesh, so every error looks small;
- the agent stops early and reports the tolerance as met.

None of these announce themselves. **Something independent has to check.**

The verifier pattern



A verifier is just another agent. What makes it useful is what it is *denied*:

- It does not see how the answer was produced — only the claim.
- It has its **own tools**, ideally sharing no code path with A's.
- Its system prompt rewards finding problems, not agreeing.

If A and B share the buggy solver, B will happily confirm the bug. Independence is a property of the *tools*, not the prompt.

Two verifiers we actually run

Verifier 1 — manufactured solutions (for the FEM task).
Pick a smooth u_{exact}, compute $f = -\Delta u_{\text{exact}}$ symbolically, solve with $u_h = u_{\text{exact}}$ on $\partial\Omega$, and measure the error. For smooth u_{exact} and P1 elements the rates are known: $\|u - u_h\|_{L^2} = O(h^2)$, $|u - u_h|_{H^1} = O(h)$.
The agent *chooses* u_{exact} — which is itself a small test of whether it understands the problem. A verifier that picks $u = x + y$ has proven nothing, since P1 reproduces it exactly.

Verifier 2 — an oracle integrator (for the projectile task).
The simulator uses `odeint` at default tolerance. The verifier re-integrates the claimed optimum with `solve_ivp(method="RK45", rtol=1e-10)` and reports the discrepancy — catching the case where the "optimum" is an artifact of integration error rather than physics.

Both are ordinary numerical-methods practice. The agent framing changes nothing about what makes a result trustworthy.

Where verification belongs

Recall the LangGraph exemplar. This is why it was the exemplar.

Verification as...	**What you get**
a line in the system prompt	a suggestion, followed most of the time
a tool the agent may call	used when the agent feels uncertain
a node in a graph	**runs every time, or the result never ships**

The pattern and the reliability requirement are the same decision. Choosing LangGraph over ReAct for the production FEM workflow *is* choosing to make verification non-optional.

This is the thread that connects the framework zoo to the engineering: you don't pick a framework by taste, you pick it by which guarantee you need.

Hands-on: what you'll build

Notebook `01_agentic_patterns.ipynb` — the zoo

- §1–§5: each pattern, twice (framework syntax + ∼15 lines from scratch)
- §6: **ReAct vs. MPC head-to-head** on the projectile, same budget
- §7: `scipy.optimize` baseline — the result that keeps us honest

Notebook `02_agent_hackathon.ipynb` — the FEM hackathon

- **A1**: uniform-refinement agent (`uniform_refine` only)
- **A2**: adaptive agent (Dörfler marking, $\theta = 0.5$)
- **A3**: rate-checking agent (adapts θ when the rate drops)
- **V**: manufactured-solution verifier

Every TODO has a worked solution one cell below. **Use it.** Reading a good system prompt is the fastest way to learn to write one.

Stretch goals

If you finish early, or want somewhere to go afterward:

- **Swap the backend.** Point the shim at Anthropic or OpenAI and see what changes in the tool-call round trip. (Answer: less than you'd expect.)
- **Break a tool on purpose.** Introduce a sign error in the load vector. Does your verifier catch it? Does the convergence plot?
- **Wire the FEM tools as a real graph.** Take the A2 agent and put the verifier on a conditional edge so a FAIL routes to further refinement.
- **Adversarial verifier.** Rewrite the verifier's system prompt to reward finding problems. Compare its pass rate against the neutral one.
- **Serve the FEM tools over MCP** and drive them from a client that isn't this notebook.

Taking it further

Agent patterns

- ReAct — Yao et al. 2022, `https://arxiv.org/abs/2210.03629`
- LangGraph — `https://langchain-ai.github.io/langgraph/`
- AutoGen — `https://microsoft.github.io/autogen/`
- Model Context Protocol — `https://modelcontextprotocol.io`
- smolagents (minimal framework) — `https://huggingface.co/docs/smolagents`

Agents doing science

- FunSearch — *Nature* 625, 2024; AlphaEvolve — DeepMind, 2025
- ChemCrow — `https://arxiv.org/abs/2304.05376`
- Coscientist — *Nature* 624, 2023
- Sakana AI Scientist — `https://sakana.ai/ai-scientist/`

Numerics

- scikit-fem — `https://scikit-fem.readthedocs.io/`
- Dörfler, *SIAM J. Numer. Anal.* 33(3), 1996

Let's go.

Open `01_agentic_patterns.ipynb` and run Part 0.

Materials: `github.com/natrask/AESCAPE`